

O Codificador Limpo:

Um Código de Conduta para Programadores Profissionais



Capítulo 6: Praticando



PET/ADS/IFSP

**Este material foi desenvolvido pelo grupo
PET/ADS do IFSP São Carlos**

Considerações sobre o ato de praticar em programação:

Praticar **não necessariamente** precisa ser algo complexo:

- Um simples programa que imprime “Hello World” pode ser considerado uma prática.

A prática **refina** nossas habilidades de adaptação para diversos programas:

- Um código prático bem sucedido pode ser interpretado como um selo de qualidade.

A história de praticar códigos:

A essência de “codar” antigamente **segue a mesma** nos dias atuais:

- Mesmo com diversas novas tecnologias, continuamos escrevendo condicionais, laços e atribuições, assim como antigamente.

Compilações demoravam diversos minutos e hoje demoram **segundos** :

- As máquinas estão cada vez mais velozes, o que evidencia mais ainda a necessidade de sermos ágeis, o que exige muita prática.

Métodos para praticar:

Praticar **individualmente** ou em **grupos** :

- Conhecido também como “Coding Dojo”, a prática contém diversos métodos que se encaixam com diferentes públicos e necessidades.

Desafio pessoal (Kata):

- Realizar uma mesma tarefa com exatidão de maneira consciente condiciona o corpo e mente a reagir rápido e com exatidão. Apresenta solução passo a passo ao grupo.

Veja na prática: [https://codingdojo.org/
http://butunclebob.com/ArticleS.UncleBob.TheBowlingGameKata
https://katas.softwarecraftsmanship.org/?p=71](https://codingdojo.org/http://butunclebob.com/ArticleS.UncleBob.TheBowlingGameKatahttps://katas.softwarecraftsmanship.org/?p=71)

Métodos para praticar:

Desafios em dupla (Wasa):

- Um membro descreve um teste e o outro deve escrever um código que faça-o passar. Na sequência, se invertem os papéis. A competição estimula o desenvolvimento de habilidades.

Desafios em grupo (Randori):

- Similar ao Wasa, porém de maneira mais descontraída e divertida. Em grupo, cada um escreve um teste e a pessoa seguinte deve fazê-lo passar. O processo é repetido entre todos..

Veja na prática: <https://wiki.c2.com/?PairProgrammingPingPongPattern>

O problema de não praticar:

Limitação da variedade de tecnologias (stack):

- Quando não praticamos, podemos ficar presos nas mesmas linguagens, ferramentas e domínios.

Difícil adaptação às mudanças no mercado:

- Praticar estimula nossa capacidade de nos adaptar e conhecer novas tecnologias. Sem ela, ficamos despreparados às mudanças.

Formas de se manter à frente:

Contribuir para projetos **open-source** :

- A pluralidade de projetos *open-source* traz, além do desafio, uma oportunidade de contribuir em projetos que possuem utilidade.

Se **desassocie** do trabalho:

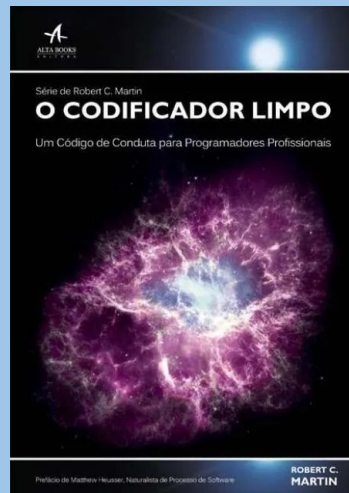
- A prática não precisa estar relacionada diretamente ao trabalho, temos a liberdade de escolher algo novo, desafiador ou simplesmente mais divertido.

Sobre o autor

Robert C. “Uncle Bob” Martin é desenvolvedor e consultor de software desde 1990. Ele é o fundador e o presidente da Object Mentor, Inc., uma equipe de consultores experientes que orientam seus clientes no mundo todo em C++, Java, C#, Ruby, OO, Padrões de Projeto, UML, Metodologias Agile e eXtreme Programming.

Este conjunto de slides foi elaborado a partir da obra:

MARTIN, Robert C. **O codificador limpo:**
Um código de conduta para programadores
profissionais. 1. ed. Rio de Janeiro: Alta
Books, 2012.



A equipe



Pedro Neo, Autor

Aluno do primeiro período,
é bolsista do PET/ADS desde
novembro de 2025.



Mariana Sônego, Revisor

Aluna do primeiro período,
é bolsista do PET/ADS desde
novembro de 2025.



Lucas Oliveira, Tutor

Professor de Computação, é
tutor do PET/ADS desde
janeiro de 2023.